

React Router

Введение в React Router

React Router — это стандартная библиотека маршрутизации для React. Она позволяет создавать навигацию в одностраничных приложениях (SPA), синхронизируя пользовательский интерфейс с URL-адресом браузера. React Router позволяет реализовать динамическую маршрутизацию, которая происходит непосредственно в приложении, а не на сервере.

Установка React Router

Для установки React Router в ваш проект используйте npm или yarn:

```
npm install react-router-dom

# или с использованием yarn
yarn add react-router-dom
```

Основные компоненты React Router

BrowserRouter

BrowserRouter использует HTML5 History API для синхронизации UI с URL-адресом. Это наиболее распространенный тип роутера для современных веб-приложений.

```
import { BrowserRouter } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      {/* Здесь размещается содержимое приложения */}
    </BrowserRouter>
  );
}
```

HashRouter

HashRouter использует хэш в URL (например, example.com/#/about) для синхронизации UI с URL-адресом. Это полезно для старых браузеров, которые не поддерживают History API, или когда у вас нет контроля над конфигурацией сервера.

```
import { HashRouter } from 'react-router-dom';
```

```
function App() {
  return (
    <HashRouter>
      {/* Здесь размещается содержимое приложения */}
    </HashRouter>
  );
}
```

Routes и Route

Routes и **Route** используются для определения маршрутов в приложении. **Routes** заменил **Switch** в React Router v6.

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Link

Link используется для навигации между страницами без перезагрузки. Он рендерит доступный тег **<a>** с правильным **href**.

```
import { Link } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <ul>
        <li>
          <Link to="/">Главная</Link>
        </li>
        <li>
          <Link to="/about">О нас</Link>
        </li>
        <li>
          <Link to="/contact">Контакты</Link>
        </li>
      </ul>
    </nav>
  );
}
```

```
    </nav>
  );
}
```

NavLink

NavLink — это специальная версия **Link**, которая добавляет стили к активному элементу. Это полезно для навигационных меню, где вы хотите выделить текущую страницу.

```
import { NavLink } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <ul>
        <li>
          <NavLink
            to="/"
            className={({ isActive }) => isActive ? "active" : ""}
          >
            Главная
          </NavLink>
        </li>
        <li>
          <NavLink
            to="/about"
            className={({ isActive }) => isActive ? "active" : ""}
          >
            О нас
          </NavLink>
        </li>
        <li>
          <NavLink
            to="/contact"
            className={({ isActive }) => isActive ? "active" : ""}
          >
            Контакты
          </NavLink>
        </li>
      </ul>
    </nav>
  );
}
```

Outlet

Outlet используется для рендеринга дочерних маршрутов. Это полезно для создания макетов с вложенными маршрутами.

```
import { Outlet } from 'react-router-dom';

function Layout() {
  return (
    <div>
      <header>
        <Navigation />
      </header>
      <main>
        <Outlet /> {/* Здесь будут отображаться вложенные маршруты */}
      </main>
      <footer>
        <p>© 2023 Моё приложение</p>
      </footer>
    </div>
  );
}
```

Вложенные маршруты

React Router v6 упрощает создание вложенных маршрутов. Вложенные маршруты определяются как дочерние элементы родительского маршрута.

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route index element={<Home />} /> {/* Индексный маршрут */}
          <Route path="about" element={<About />} />
          <Route path="contact" element={<Contact />} />
          <Route path="products" element={<Products />}>
            <Route index element={<ProductsList />} />
            <Route path=":id" element={<ProductDetails />} />
          </Route>
          <Route path="*" element={<NotFound />} /> {/* Маршрут для 404 */}
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

Параметры URL

React Router позволяет определять динамические сегменты URL с помощью параметров. Для доступа к параметрам используется хук `useParams`.

```
import { useParams } from 'react-router-dom';

function ProductDetails() {
  const { id } = useParams();
  return <div>Детали продукта с ID: {id}</div>;
}
```

Программная навигация

Для программной навигации (например, после отправки формы) можно использовать хук `useNavigate`.

```
import { useNavigate } from 'react-router-dom';

function LoginForm() {
  const navigate = useNavigate();

  const handleSubmit = (event) => {
    event.preventDefault();
    // Логика авторизации
    navigate('/dashboard'); // Перенаправление на страницу dashboard
  };

  return (
    <form onSubmit={handleSubmit}>
      {/* Поля формы */}
      <button type="submit">Войти</button>
    </form>
  );
}
```

Доступ к параметрам запроса

Для доступа к параметрам запроса (query parameters) используется хук `useSearchParams`.

```
import { useSearchParams } from 'react-router-dom';

function ProductsFilter() {
  const [searchParams, setSearchParams] = useSearchParams();
  const category = searchParams.get('category');
  const minPrice = searchParams.get('minPrice');

  const handleCategoryChange = (event) => {
    const category = event.target.value;
    if (category) {
      searchParams.set('category', category);
    }
  };
}
```

```

    } else {
      searchParams.delete('category');
    }
    setSearchParams(searchParams);
  };

  return (
    <div>
      <h2>Фильтр продуктов</h2>
      <div>
        <label>
          Категория:
          <select value={category || ''} onChange={handleCategoryChange}>
            <option value="">Все</option>
            <option value="electronics">Электроника</option>
            <option value="clothing">Одежда</option>
            <option value="books">Книги</option>
          </select>
        </label>
      </div>
      { /* Другие фильтры */ }
    </div>
  );
}

```

Доступ к текущему местоположению

Для доступа к текущему местоположению используется хук `useLocation`.

```

import { useLocation } from 'react-router-dom';

function CurrentLocation() {
  const location = useLocation();

  return (
    <div>
      <p>Текущий путь: {location.pathname}</p>
      <p>Параметры запроса: {location.search}</p>
      <p>Хэш: {location.hash}</p>
      <p>Состояние: {JSON.stringify(location.state)}</p>
    </div>
  );
}

```

Защищенные маршруты

Часто требуется ограничить доступ к определенным маршрутам только для авторизованных пользователей. В React Router v6 это можно реализовать с помощью компонента-обертки.

```

import { Navigate, useLocation } from 'react-router-dom';

function RequireAuth({ children }) {
  const location = useLocation();
  const isAuthenticated = checkIfUserIsAuthenticated(); // Ваша функция
  проверки аутентификации

  if (!isAuthenticated) {
    // Перенаправление на страницу входа с сохранением исходного URL
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  return children;
}

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Layout />} />
        <Route index element={<Home />} />
        <Route path="login" element={<Login />} />
        <Route
          path="dashboard"
          element={
            <RequireAuth>
              <Dashboard />
            </RequireAuth>
          }
        />
        <Route
          path="profile"
          element={
            <RequireAuth>
              <Profile />
            </RequireAuth>
          }
        />
      </Routes>
    </BrowserRouter>
  );
}

```

Ленивая загрузка компонентов

Для оптимизации производительности можно использовать ленивую загрузку компонентов с помощью `React.lazy` и `Suspense`.

```
import React, { Suspense, lazy } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';

const Home = lazy(() => import('./Home'));
const About = lazy(() => import('./About'));
const Contact = lazy(() => import('./Contact'));

function App() {
  return (
    <BrowserRouter>
      <Suspense fallback={<div>Загрузка...</div>}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/contact" element={<Contact />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```

Различия между версиями React Router

React Router v6 внес значительные изменения по сравнению с v5. Вот некоторые из основных отличий:

1. **Switch** заменен на **Routes**
2. **component** и **render** props заменены на **element**
3. Вложенные маршруты теперь определяются как дочерние элементы
4. **useHistory** заменен на **useNavigate**
5. **Redirect** заменен на **Navigate**

Заключение

React Router — мощная библиотека для реализации маршрутизации в React-приложениях. Она предоставляет декларативный способ определения маршрутов, поддерживает вложенные маршруты, параметры URL, программную навигацию и многое другое.

При разработке React-приложений с несколькими страницами React Router является практически незаменимым инструментом, который значительно упрощает создание интуитивно понятной навигации и улучшает пользовательский опыт.